

Sistem Layanan REST API Terdistribusi Sederhana

Pembagian Beban Round-Robin dan Failover
Python Flask • NGINX • SQLite

Iryan Tegar

NIM:

Kelas:

Program Studi Informatika
Universitas Muhammadiyah Makassar
2026

Ringkasan

Purnarupa ini menjalankan satu aplikasi REST API dalam dua proses pada port 5001 dan 5002. NGINX pada port 8080 membagi permintaan ke kedua proses. Keduanya memakai satu database SQLite, sehingga data yang dibuat oleh Server A dapat dibaca Server B. Pengujian menunjukkan distribusi 5:5, data bersama konsisten, dan 10 dari 10 request berhasil melalui B setelah A dihentikan.

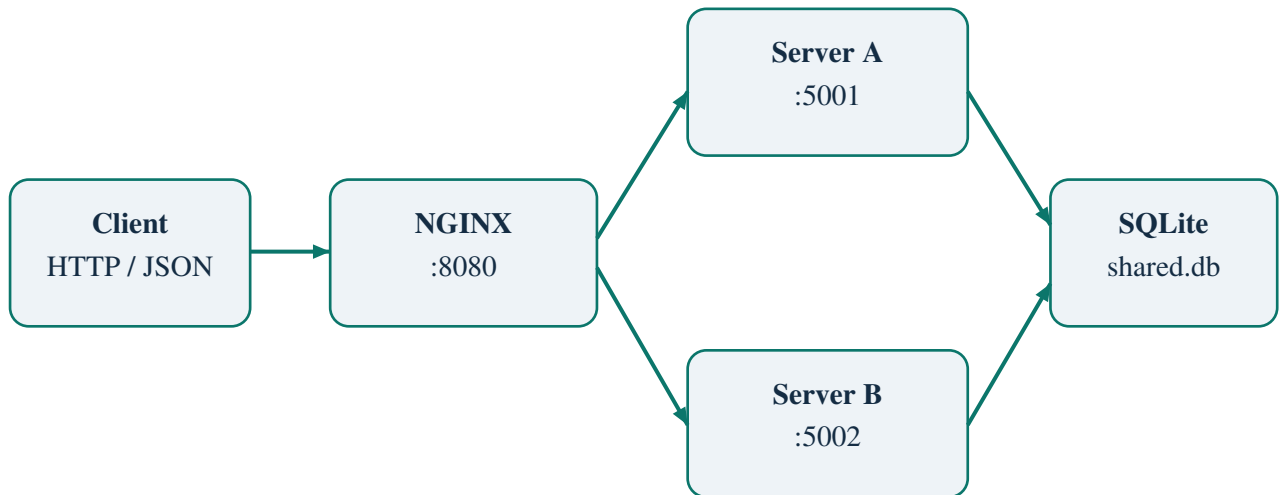
Komponen penilaian	Bobot	Implementasi
REST API dan identitas server	20%	GET/POST /api/data; server_id dan port
Replikasi layanan	20%	A:5001 dan B:5002 aktif bersamaan
NGINX load balancer	20%	Round-robin melalui :8080
Shared data	20%	Satu file SQLite data/shared.db
Distribusi dan failover	20%	10 request normal + 10 setelah A mati

Identitas NIM dan kelas belum diberikan sehingga disediakan kolom isian. Laporan berisi hasil uji di lingkungan Linux eksekusi, bukan klaim pengujian pada laptop pribadi.

1. Rancangan dan aliran data

Tujuan percobaan adalah membuktikan pembagian beban permintaan dan keberlanjutan layanan saat satu proses aplikasi berhenti, dengan arsitektur sederhana tanpa message broker. Replikasi dilakukan pada proses aplikasi; database tetap satu berkas bersama.

Dua instance aplikasi mengakses satu berkas data yang sama



Panah menunjukkan arah request/akses data; respons kembali melalui jalur yang sama.

Gambar 1. Diagram blok Client - NGINX - Server A/B - database bersama.

Komponen	Alamat / berkas	Tanggung jawab
Client	http://localhost:8080	Mengirim request HTTP melalui load balancer
NGINX	127.0.0.1:8080	Memilih upstream dan melakukan retry yang sesuai
Server A	127.0.0.1:5001	Menjalankan server.py dengan ID A
Server B	127.0.0.1:5002	Menjalankan server.py dengan ID B
Database	data/shared.db	Menyimpan data yang dibaca kedua proses

Alur normal dan ketika gagal

Pada kondisi normal, client mengirim GET /api/data ke NGINX. Dengan bobot kedua server sama, NGINX menggunakan round-robin sebagai metode default (NGINX, n.d.-a). Server terpilih membaca SQLite lalu mengembalikan JSON berisi pesan, data, server_id, dan port. Respons diteruskan NGINX ke client.

Ketika A dihentikan, percobaan koneksi NGINX ke A gagal. Dengan max_fails=1, fail_timeout=30s, dan retry ke upstream berikutnya, request GET dialihkan ke B. Database tidak ikut dihentikan karena berbentuk file bersama di host yang sama.

Lingkungan pengujian

Pengujian menggunakan Python 3.12, Flask 3.1.3, NGINX 1.28.0, dan SQLite bawaan Python. Semua layanan memakai loopback. NGINX dikompilasi lokal; sandbox memerlukan tambahan runtime user root dan master_process off. Konfigurasi distribusi dan failover tetap memakai nginx.conf yang disertakan. Tambahan runtime itu tidak diperlukan pada laptop biasa.

2. REST API dan shared data

server.py yang sama dijalankan dengan argumen ID dan port berbeda. GET /api/data mengembalikan HTTP 200; POST menerima JSON dengan name berisi 1-100 karakter dan mengembalikan HTTP 201. Input tidak valid ditolak dengan HTTP 400. Kutipan berikut menunjukkan logika intinya; kode lengkap tersedia dalam ZIP.

```
def reply(message, status=200, **extra):
    return jsonify(message=message, server_id=server_id,
                    port=port, **extra), status

@app.route("/api/data", methods=["GET", "POST"])
def data():
    with closing(connect_db()) as db:
        if request.method == "GET":
            rows = db.execute("SELECT * FROM items ORDER BY id").fetchall()
            return reply("Data bersama berhasil dibaca",
                        data=[dict(row) for row in rows])
        body = request.get_json(silent=True)
        name = body.get("name") if isinstance(body, dict) else None
        if not isinstance(name, str) or not 1 <= len(name.strip()) <= 100:
            return reply("name wajib berupa teks 1-100 karakter", 400)
        with db:
            cur = db.execute(
                "INSERT INTO items(name, created_by) VALUES (?, ?)",
                (name.strip(), server_id))
            row = db.execute("SELECT * FROM items WHERE id = ?",
                            (cur.lastrowid,)).fetchone()
            return reply("Data berhasil ditambahkan", 201, data=dict(row))
```

Skema database (schema.sql)

```
CREATE TABLE IF NOT EXISTS items (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL CHECK(length(trim(name)) BETWEEN 1 AND 100),
    created_by TEXT NOT NULL,
    created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now'))
);
```

Path database dihitung dari lokasi server.py, sehingga kedua proses membuka file yang sama meskipun terminal memiliki direktori kerja berbeda. init_db.py membuat tabel secara idempoten tanpa menghapus data. SQL memakai parameter (?) dan transaksi commit sebelum POST mengirim respons berhasil.

SQLite memakai mode WAL dan timeout koneksi lima detik. WAL memungkinkan pembaca dan penulis berjalan bersamaan, tetapi hanya satu penulis aktif pada satu waktu; seluruh proses harus berada pada host yang sama (SQLite, n.d.). Demonstrasi ini tidak memakai replikasi database atau file di network filesystem.

3. Konfigurasi NGINX

Berkas `nginx.conf` berikut dijalankan dari folder proyek. Satu worker dipilih untuk memudahkan pengamatan urutan round-robin dalam pengujian berurutan tanpa trafik tambahan.

```
worker_processes 1;
pid logs/nginx.pid;
error_log logs/error.log warn;

events {
    worker_connections 256;
}

http {
    default_type application/json;
    log_format upstream_log '$time_iso8601 "$request" status=$status '
        'upstream=$upstream_addr upstream_status=$upstream_status';
    access_log logs/access.log upstream_log;
    client_body_temp_path logs/client_body;
    proxy_temp_path logs/proxy_temp;

    upstream backend {
        # Round-robin adalah metode default NGINX.
        server 127.0.0.1:5001 max_fails=1 fail_timeout=30s;
        server 127.0.0.1:5002 max_fails=1 fail_timeout=30s;
    }

    server {
        listen 127.0.0.1:8080;
        server_name localhost;
        client_max_body_size 16k;

        location / {
            proxy_pass http://backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_connect_timeout 1s;
            proxy_read_timeout 5s;
            proxy_next_upstream error timeout http_502 http_503 http_504;
            proxy_next_upstream_tries 2;
            add_header X-Upstream-Addr $upstream_addr always;
        }
    }
}
```

Parameter utama. `max_fails=1` menandai upstream gagal setelah kegagalan yang memenuhi kriteria. `fail_timeout=30s` juga menentukan masa penghindaran upstream. `proxy_next_upstream` mengizinkan retry pada error yang dipilih, sedangkan `proxy_next_upstream_tries 2` membatasi total percobaan upstream (NGINX, n.d.-a, n.d.-b).

Pemeriksaan ini bersifat pasif: NGINX mengetahui kegagalan dari trafik yang sedang berjalan. Endpoint `/health` tersedia untuk pemeriksaan manual, tetapi tidak dipolling secara otomatis oleh konfigurasi ini.

4. Menjalankan dan menguji data bersama

Pasang Python, venv, dan NGINX di Ubuntu atau WSL Ubuntu. Jalankan seluruh terminal dari folder proyek yang sama. Rincian instalasi dan penanganan port tersedia dalam README.md.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python init_db.py
# Terminal A dan B (aktifkan .venv di masing-masing terminal):
python server.py --id A --port 5001
python server.py --id B --port 5002
# Terminal C:
nginx -p "$PWD/" -c nginx.conf -t
nginx -p "$PWD/" -c nginx.conf -g 'daemon off;'
# Terminal D, jalankan distribusi lebih dahulu:
python uji.py round-robin
python uji.py shared
```

Uji shared menambahkan data langsung melalui A:5001, kemudian membacanya langsung melalui B:5002. Pemilihan port langsung memastikan pembuat dan pembaca diketahui; uji distribusi dan failover tetap melalui NGINX:8080.

Uji data bersama • tulis A, baca B

Penampil log pengujian nyata | Python Flask + NGINX + SQLite

```
$ python uji.py shared
Waktu UTC: 2026-09-21T23:44:14+00:00
POST :5001/api/data | HTTP 201 | server=A
{"created_at": "2026-09-21T23:44:14.399Z", "created_by": "A", "id": 2, "name": "Data-uji-234414394981"}
GET :5002/api/data | HTTP 200 | server=B
{"created_at": "2026-09-21T23:44:14.399Z", "created_by": "A", "id": 2, "name": "Data-uji-234414394981"}
LULUS: ID, isi, pembuat, dan waktu data sama pada A dan B.
```

Sumber: shared.txt • isi log ditampilkan tanpa mengubah hasil.

Gambar 2. Screenshot penampil log asli: POST dilayani A, lalu data yang sama dibaca B.

Hasil: POST berstatus 201 dan GET berstatus 200. Baris dengan ID 2, nama Data-uji-234414394981, created_by=A, serta timestamp yang sama muncul dalam respons B. Ini membuktikan bahwa commit dari A dapat langsung dibaca oleh B. Database dalam paket juga menyimpan satu baris dari pengujian awal.

5. Bukti distribusi 10 request

Saat A dan B aktif bersamaan, perintah python uji.py round-robin mengirim tepat 10 request GET /api/data secara berurutan ke localhost:8080. Skrip memeriksa kode status, menghitung jumlah respons tiap server, dan memastikan identitas server berubah pada setiap pasangan request.

Uji distribusi • 10 request melalui NGINX

Penampil log pengujian nyata | Python Flask + NGINX + SQLite

```
$ python uji.py round-robin
Waktu UTC: 2026-09-21T23:44:14+00:00
01 | HTTP 200 | Server A | port=5001 | upstream=127.0.0.1:5001
02 | HTTP 200 | Server B | port=5002 | upstream=127.0.0.1:5002
03 | HTTP 200 | Server A | port=5001 | upstream=127.0.0.1:5001
04 | HTTP 200 | Server B | port=5002 | upstream=127.0.0.1:5002
05 | HTTP 200 | Server A | port=5001 | upstream=127.0.0.1:5001
06 | HTTP 200 | Server B | port=5002 | upstream=127.0.0.1:5002
07 | HTTP 200 | Server A | port=5001 | upstream=127.0.0.1:5001
08 | HTTP 200 | Server B | port=5002 | upstream=127.0.0.1:5002
09 | HTTP 200 | Server A | port=5001 | upstream=127.0.0.1:5001
10 | HTTP 200 | Server B | port=5002 | upstream=127.0.0.1:5002
Jumlah: A=5, B=5; sukses=10/10
LULUS: dua server bergantian melayani 10 request.
```

Sumber: round-robin.txt • isi log ditampilkan tanpa mengubah hasil.

Gambar 3. Screenshot penampil log asli dari 10 request melalui NGINX.

Ukuran pengamatan	Hasil
Urutan server	A, B, A, B, A, B, A, B, A, B
Respons Server A	5 request (50%)
Respons Server B	5 request (50%)
Respons berhasil	10/10, seluruhnya HTTP 200
Bukti pendukung	bukti/round-robin.txt dan bukti/access.log

Distribusi sesuai round-robin pada skenario terkontrol ini. Header X-Upstream-Addr memperlihatkan port tujuan yang sama dengan identitas JSON. Log akses NGINX juga merekam alamat upstream. Hasil ini membuktikan pergiliran request, bukan peningkatan throughput atau kapasitas maksimum; benchmark performa tidak menjadi bagian percobaan.

Pengujian dilakukan tanpa request tambahan ke port 8080. Pada penggunaan browser, request favicon atau trafik lain dapat menyela urutan yang terlihat oleh seorang client. Oleh sebab itu, verifikasi dilakukan dengan skrip request berurutan.

6. Bukti failover setelah A dihentikan

Server A dihentikan secara manual dari terminal interaktif dengan kill -TERM terhadap PID proses A yang telah dicatat. B dan NGINX tetap berjalan. Skrip failover memeriksa bahwa koneksi langsung ke port 5001 gagal, kemudian mengirim 10 request GET melalui port 8080.

Uji failover • Server A dihentikan manual

Penampil log pengujian nyata | Python Flask + NGINX + SQLite

```
Perintah manual: kill -TERM 5
Server A dihentikan dari terminal interaktif.

$ python uji.py failover
Waktu UTC: 2026-09-21T23:44:24+00:00
Cek langsung :5001 -> koneksi gagal (Server A mati).
01 | HTTP 200 | Server B | port=5002 | upstream=127.0.0.1:5001, 127.0.0.1:5002
02 | HTTP 200 | Server B | port=5002 | upstream=127.0.0.1:5002
03 | HTTP 200 | Server B | port=5002 | upstream=127.0.0.1:5002
04 | HTTP 200 | Server B | port=5002 | upstream=127.0.0.1:5002
05 | HTTP 200 | Server B | port=5002 | upstream=127.0.0.1:5002
06 | HTTP 200 | Server B | port=5002 | upstream=127.0.0.1:5002
07 | HTTP 200 | Server B | port=5002 | upstream=127.0.0.1:5002
08 | HTTP 200 | Server B | port=5002 | upstream=127.0.0.1:5002
09 | HTTP 200 | Server B | port=5002 | upstream=127.0.0.1:5002
10 | HTTP 200 | Server B | port=5002 | upstream=127.0.0.1:5002
Jumlah: A=0, B=10; sukses=10/10
LULUS: 10 request tetap berhasil melalui Server B.
```

Sumber: failover.txt • Isi log ditampilkan tanpa mengubah hasil.

Gambar 4. Screenshot penampil log asli: penghentian A dan 10 respons sukses dari B.

Hasil: A=0, B=10, seluruh request menerima HTTP 200. Request pertama mencatat upstream A kemudian B. Log NGINX menunjukkan upstream_status=502, 200, tetapi status akhir ke client adalah 200. Dengan demikian, kegagalan koneksi ke A ditangani oleh retry ke B, bukan dengan memindahkan URL client secara manual.

Setelah 30 detik, NGINX dapat mencoba A kembali menggunakan trafik nyata. Jika A tetap mati, request GET dapat dicoba ulang ke B. Uji ini mencakup satu rangkaian 10 request setelah proses A dihentikan; tidak membuktikan ketiadaan gangguan untuk seluruh jenis kegagalan atau durasi operasi.

Untuk mengulang di laptop: tekan Ctrl+C pada terminal Server A, lalu jalankan python uji.py failover dari terminal pengujian. Jalankan kembali A dan tunggu minimal 30 detik jika ingin mengulang distribusi normal.

7. Kesimpulan dan kelengkapan luaran

Purnarupa memenuhi lima komponen implementasi pada soal. Satu aplikasi REST API direplikasi ke dua port, NGINX membagi 10 request secara bergantian dengan rasio 5:5, dan database bersama menjaga akses data antarproses. Setelah A dihentikan manual, 10 request selanjutnya tetap berhasil melalui B. Konsep load balancing dan ketersediaan layanan pada kegagalan satu instance telah dibuktikan.

Batasan hasil

Ketersediaan yang diuji hanya meliputi kegagalan satu proses aplikasi. NGINX, database, dan komputer masih menjadi titik kegagalan tunggal. Flask development server digunakan untuk praktikum lokal; sistem belum dirancang sebagai deployment produksi. Pengujian failover memakai GET. Retry terhadap POST yang sudah dikirim tidak dipaksakan karena berisiko menghasilkan operasi ganda (NGINX, n.d.-b).

Luaran yang diminta	Berkas / lokasi dalam paket
Kode server dan dependensi	server.py; requirements.txt
Konfigurasi load balancer	nginx.conf
Skrip dan data bersama	schema.sql; init_db.py; data/shared.db
Skrip pengujian dan panduan	uji.py; demo_otomatis.py; README.md
Diagram blok	Laporan halaman 2
Screenshot distribusi dan failover	bukti/round-robin.png; bukti/failover.png
Bukti shared data dan log mentah	bukti/shared.png; *.txt; access.log; error.log
Laporan PDF minimal 5 halaman	Laporan ini terdiri dari 8 halaman

Transparansi bukti. Screenshot dibuat dengan menangkap tampilan browser yang memuat log pengujian asli, bukan gambar hasil rekaan atau foto terminal laptop pengguna. Berkas log mentah dan halaman HTML penampil disertakan. Pengujian akhir tercatat pada 21 September 2026 pukul 23:44 UTC, setara 22 September 2026 pukul 07:44 WITA.

Daftar pustaka

NGINX. (n.d.-a). *Using nginx as HTTP load balancer*.

https://nginx.org/en/docs/http/load_balancing.html

NGINX. (n.d.-b). *Module ngx_http_proxy_module*.

https://nginx.org/en/docs/http/ngx_http_proxy_module.html

SQLite. (n.d.). *Write-ahead logging*.

<https://www.sqlite.org/wal.html>

Sumber tugas: *SOAL UJIAN SEMESTER ANTARA.pdf*, dua halaman, dokumen soal yang diberikan pengguna.